Docker para ASIR

Resumen visual para memorizar: 11 apartados clave

APARTADO 1 · pág. 1
Imagen vs Contenedor
`plantilla -> N instancias`

APARTADO 2 · pág. 2
Ciclo de vida
`create · run · start`
`stop · restart · rm`

APARTADO 3 · pág. 3
Puertos -p
`host:contenedor`
`8080:80`

APARTADO 4 · pág. 3
Bind mount -v
`carpeta compartida`
`host:contenedor`

APARTADO 5 · pág. 4
attach vs exec
`dos formas de entrar`
`en un contenedor`

APARTADO 6 · pág. 4
Ver y filtrar
`ps · image ls`
`-q · --filter · --format`

APARTADO 7 · pág. 5
Logs
`stdout · stderr`
`-f · -t · --tail`

APARTADO 8 · pág. 6
Otros comandos
`commit · cp · diff · top`
`inspect · stats · tag...`

APARTADO 9 · pág. 6
docker info
`Root Dir`
`Storage Driver`

APARTADO 10 · pág. 7
Dockerfile + build
`FROM · RUN · COPY`
`build -t · -f`

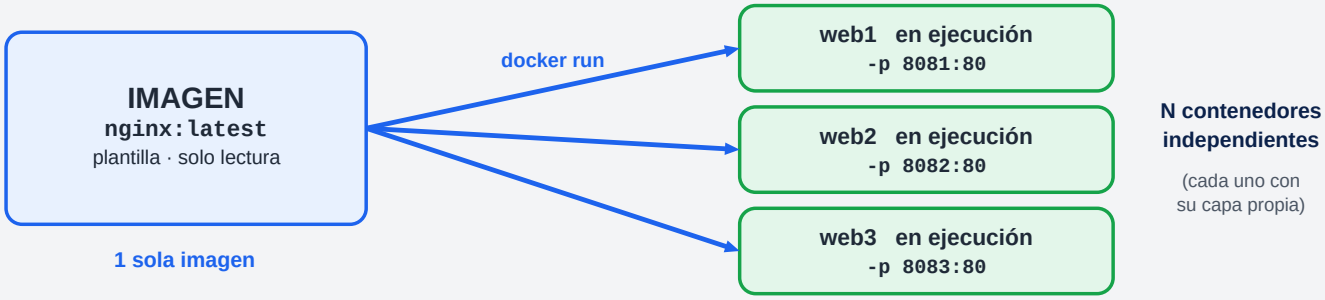
APARTADO 11 · pág. 8
Docker Compose
`docker-compose.yml`
`up -d · down -v`

Bloques de color

- Conceptos (1)
- Creación (2-5)
- Consulta (6-7)
- Avanzado (8-11)

1 Imagen vs Contenedor

CONCEPTOS



IMAGEN

Plantilla **de solo lectura** con el sistema de ficheros, la app y su configuración. Está hecha de **capas**. Se descarga (pull) de un registro como Docker Hub.

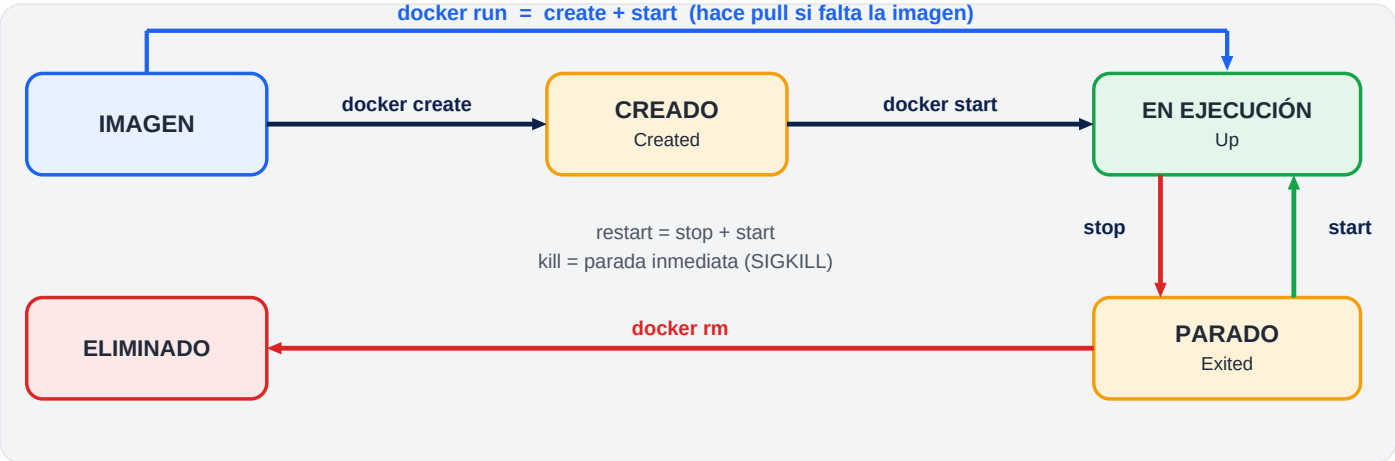
CONTENEDOR

Instancia en ejecución de una imagen: proceso aislado con su propia red y ficheros. Es **efímero**: al borrarlo se pierde lo que no esté en un volumen.

	Imagen	Contenedor
Estado	Estática, solo lectura	Viva, en ejecución
Se obtiene con	`docker pull` / `docker build`	`docker run` / `docker create`
Cantidad	1 imagen	Muchos contenedores a partir de ella
Se lista con	`docker image ls`	`docker ps -a`
Analogía	ISO / plantilla de VM	La VM encendida

2 Ciclo de vida

CREACIÓN



Comando	Qué hace
`docker create nginx`	Crea el contenedor pero **NO lo arranca** (estado *Created*).
`docker run nginx`	**create + start** en un paso. Si falta la imagen, hace *pull*. Crea un contenedor **nuevo** cada vez.
`docker start web`	Arranca un contenedor ya existente (creado o parado). Conserva sus datos.
`docker stop web`	Parada ordenada: SIGTERM y, a los 10 s, SIGKILL.
`docker restart web`	stop + start.
`docker kill web`	Parada inmediata (SIGKILL).
`docker rm web`	Elimina un contenedor **parado**. `-f` fuerza si está activo.

Opciones más usadas de docker run

Opción	Significado	Opción	Significado
`-d`	Segundo plano (detached)	`--name web`	Le pone nombre
`-it`	Interactivo + terminal	`--rm`	Se borra al pararse
`-p 8080:80`	Publica un puerto (3)	`-v ./h:/data`	Monta una carpeta (4)
`-e VAR=valor`	Variable de entorno	`--restart always`	Reinicia si cae

```
docker run -d --name web -p 8080:80 -v $(pwd)/html:/usr/share/nginx/html nginx
docker run -it --rm ubuntu bash        # prueba rápida, se borra al salir
docker stop web && docker rm web       # parar y borrar
```

3 Puertos -p

CREACIÓN

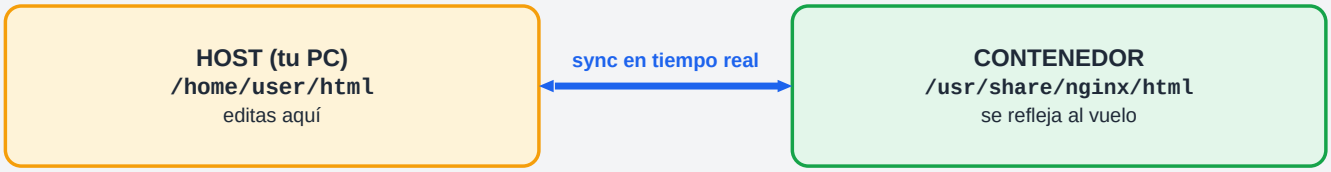
```
docker run -d -p 8080:80 nginx
docker run -d --name db \
  -p 3307:3306 \
  -e MYSQL_ROOT_PASSWORD=1234 mysql
docker run -d -P nginx    # puertos al azar
docker port web           # ver el mapeo
```

Servicio	Puerto interno
Nginx / Apache (httpd)	80
HTTPS	443
MySQL / MariaDB	3306
PostgreSQL	5432
MongoDB	27017
Redis	6379

Regla: El puerto de la **izquierda** no puede repetirse entre contenedores (dos webs: 8081 y 8082). Los puertos < 1024 son privilegiados: usa uno entre **8000 y 9000**.

4 Bind mount -v

CREACIÓN



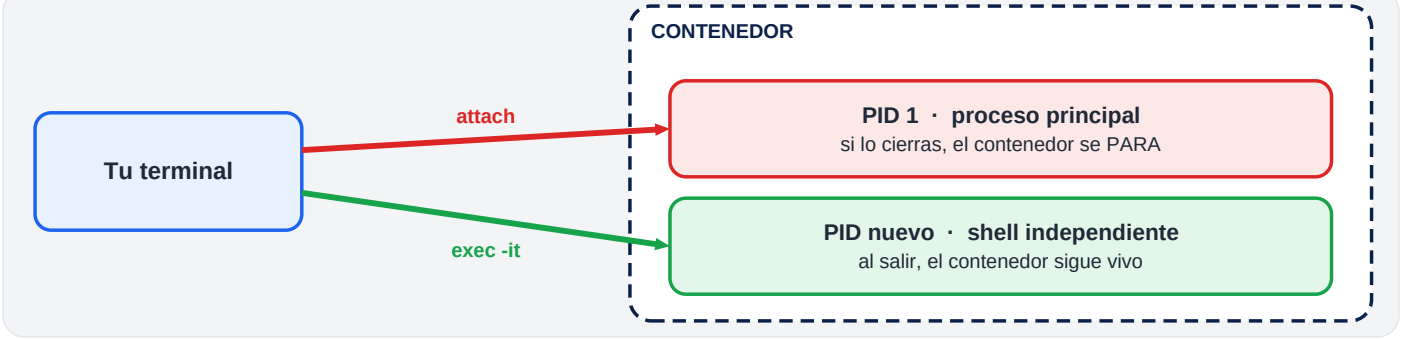

```
-v  /ruta/host : /ruta/contenedor [:ro]
     ^ lo tuyo    ^ dentro del contenedor

docker run -d -p 8080:80 \
  -v $(pwd)/html:/usr/share/nginx/html nginx
docker run -d -v ... :ro nginx   # solo lectura
docker run -d -v db_data:/var/lib/mysql mariadb
```

	Bind mount	Volumen
Origen	Ruta del host	Nombre
Lo gestiona	Tú	Docker
Uso	Desarrollo	Datos (BBDD)
Analogía	Shared folder de VM	Disco virtual

Clave: Si editas **index.html** en tu PC y recargas el navegador, el cambio se ve **sin reiniciar**. Usa ruta absoluta o **$(pwd)**. Gestión de volúmenes: `docker volume ls / create / rm`

5 attach vs exec

CREACIÓN



	docker attach	docker exec -it
Entras en	El proceso principal (PID 1)	Una shell **nueva** e independiente
Al salir (exit / Ctrl+C)	**El contenedor se CIERRA**	**El contenedor sigue vivo**
Salir sin pararlo	`Ctrl+P Ctrl+Q`	`exit`
Para qué	Ver/interactuar con la salida principal	**Administrar y depurar**
Ejemplo	`docker attach web`	`docker exec -it web sh`

Regla ASIR: Para administrar usa siempre **exec -it**. nginx:alpine no trae bash: usa **sh**. Sin **-it**, exec ejecuta un comando suelto: `docker exec web ls /`

6 Ver y filtrar

CONSULTA

salida de docker ps

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
id corto (12 chars)	**imagen** de origen	**proceso** PID 1	**cuándo** se creó	**Up / Exited** + tiempo	**mapeo -p** host->cont.	**nombre** (--name)

docker ps

Opción	Efecto
`(nada)`	Solo activos
`-a`	Todos (activos y parados)
`-q`	Solo IDs
`-aq`	IDs de todos
`-l`	El último creado
`-n 3`	Los 3 últimos
`-s`	Muestra el tamaño
`--no-trunc`	IDs completos

docker image ls (= docker images)

Opción	Efecto
`(nada)`	Imágenes locales
`-q`	Solo IDs
`-a`	Incluye intermedias
`--digests`	Muestra el digest
`--no-trunc`	IDs completos
`--filter`	Filtra (ver abajo)
`--format`	Formato propio

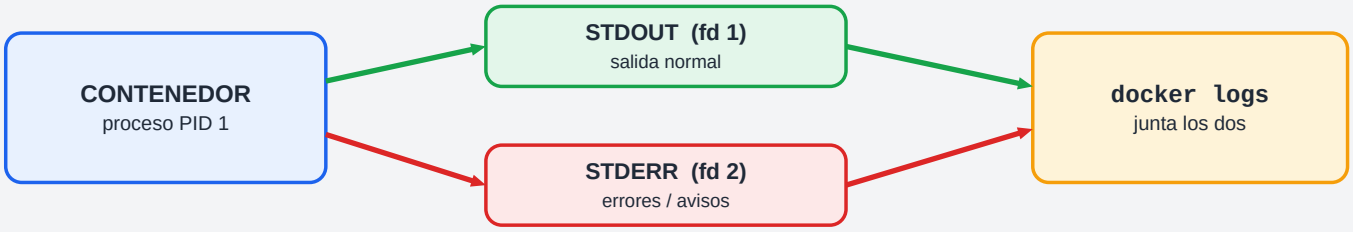
Filtros y formato

```
docker ps -a --filter "status=exited"        # parados
docker ps --filter "name=web"                # por nombre
docker ps --filter "ancestor=nginx"          # creados desde la imagen nginx
docker images --filter "dangling=true"       # imágenes sin tag
docker images --filter "reference=nginx"     # por nombre
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

docker image + subcomando	Función
`ls · inspect · history`	Listar · metadatos JSON · capas y comandos que las crearon
`pull · push · build · tag`	Descargar · subir · construir · ponerle nombre/versión
`rm (rmi) · prune`	Borrar · borrar las *dangling* (`-a`: todas las sin uso)

7 Logs

CONSULTA




```
$ docker logs -t web
2026-11-10T09:15:02.123456789Z /docker-entrypoint.sh: Configuration complete        <- stdout
2026-11-10T09:15:05.456789012Z [error] open() "/usr/share/nginx/html/x" failed      <- stderr
2026-11-10T09:15:06.000112233Z 172.17.0.1 - - "GET / HTTP/1.1" 200 615              <- stdout
```

Opción	Efecto
`-f`	Sigue en vivo (follow)
`--tail 50`	Últimas 50 líneas
`-t`	Añade fecha y hora
`--since 10m`	Solo últimos 10 min
`--until 5m`	Hasta hace 5 min

```
docker logs web
docker logs -f --tail 20 web
docker logs web 2>/dev/null   # solo stdout
docker logs web >/dev/null    # solo errores
docker logs --since 1h -t web
```

Ojo: **stdout** = salida normal, **stderr** = errores. **-t** añade la fecha. Los logs son de **un contenedor** que eliges por nombre o ID; para saber de qué imagen es, usa `docker ps` o `docker inspect`.

8 Otros comandos

AVANZADO

Comando	Qué hace	Ejemplo
`commit`	Crea una imagen nueva a partir de un contenedor	`docker commit web miweb:v1`
`cp`	Copia ficheros entre host y contenedor	`docker cp web:/etc/hosts .`
`diff`	Cambios en el sistema de ficheros (A, C, D)	`docker diff web`
`export`	Vuelca el sistema de ficheros a un .tar	`docker export web > web.tar`
`history`	Capas de una imagen y su comando	`docker history nginx`
`inspect`	Todos los metadatos en JSON (IP, volúmenes...)	`docker inspect web`
`search`	Busca imágenes en Docker Hub	`docker search mysql`
`stats`	CPU, RAM y red en vivo	`docker stats`
`tag`	Pone otro nombre/versión a una imagen	`docker tag nginx miuser/nginx:v2`
`top`	Procesos que corren dentro del contenedor	`docker top web`
`version`	Versión del cliente y del servidor	`docker version`
`system df / prune`	Espacio usado / limpieza masiva (`-a`, `--volumes`)	`docker system prune -a`

9 docker info

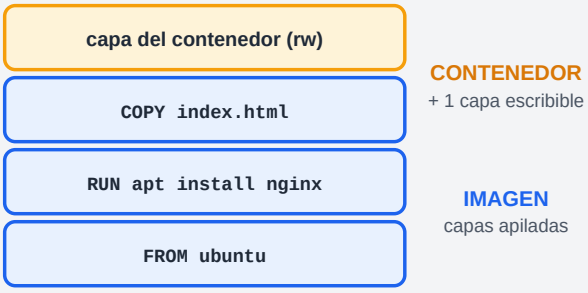
AVANZADO

Docker Root Dir

```
/var/lib/docker
```

- `containers/` — datos y logs de cada contenedor
- `image/` — metadatos de imágenes
- `overlay2/` — las CAPAS (storage driver)
- `volumes/` — volúmenes con nombre
- `network/` — redes

Storage Driver (overlay2)




```
$ docker info | grep -E "Root Dir|Storage Driver"
 Storage Driver: overlay2
 Docker Root Dir: /var/lib/docker
```

Root Dir: carpeta donde Docker guarda **toda su información** (imágenes, contenedores, volúmenes y redes).

Storage Driver: la tecnología que gestiona las **capas** (normalmente **overlay2**). Apila capas de solo lectura y añade una escribible.

10 Dockerfile + build

AVANZADO

El '.' final = CONTEXTO (carpeta actual): solo puedes copiar archivos que estén dentro de ella

Dockerfile de ejemplo

```
FROM ubuntu:24.04
RUN apt-get update && \
    apt-get install -y nginx
COPY index.html /var/www/html/
ENV TZ=Europe/Madrid
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

```
docker build -t miweb:1.0 .
docker build -f Dockerfile.dev -t miweb:dev .
docker build --no-cache -t miweb:1.0 .
docker run -d -p 8080:80 miweb:1.0
```

Instrucciones

Instrucción	Función
`FROM`	Imagen base (obligatoria)
`RUN`	Ejecuta un comando al construir
`COPY / ADD`	Copia ficheros del contexto
`WORKDIR`	Carpeta de trabajo
`ENV`	Variable de entorno
`EXPOSE`	Documenta el puerto
`CMD`	Comando al arrancar

Opción de build	Función
`-t nombre:tag`	Nombre y versión de la imagen
`-f fichero`	Usa otro Dockerfile
`--no-cache`	Reconstruye sin usar caché
`--build-arg K=V`	Pasa una variable al build
`. (contexto)`	Carpeta actual: lo que puede copiar el build

11 Docker Compose

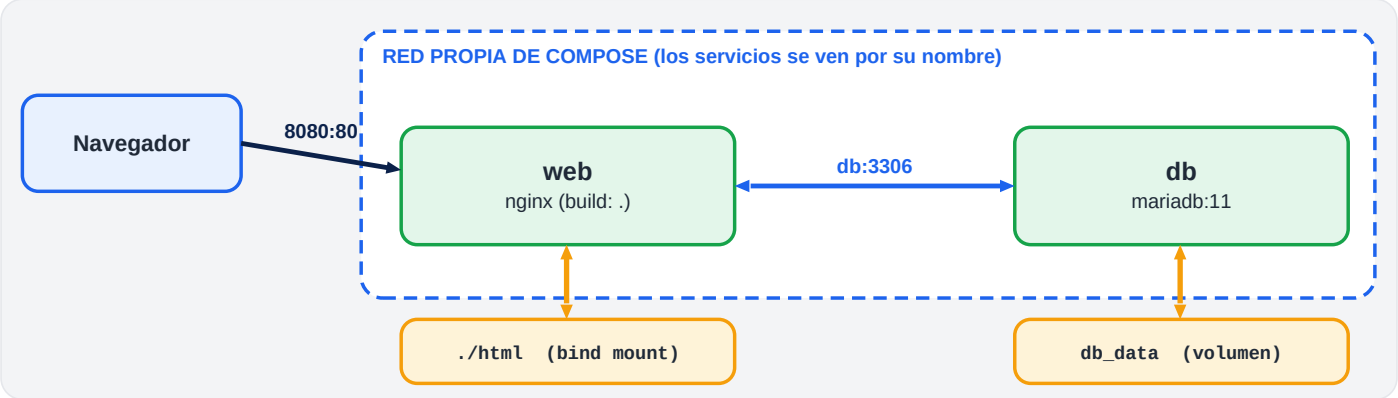
AVANZADO

docker-compose.yml

```
services:
  web:
    build: .
    ports:
      - "8080:80"
    volumes:
      - ./html:/usr/share/nginx/html
    depends_on:
      - db
  db:
    image: mariadb:11
    environment:
      MARIADB_ROOT_PASSWORD: root1234
    volumes:
      - db_data:/var/lib/mysql

volumes:
  db_data:
```

Claves principales

Clave	Función
`services`	Lista de contenedores
`image / build`	Imagen o Dockerfile
`ports`	host:contenedor
`volumes`	Bind mount o volumen
`environment`	Variables (ENV)
`depends_on`	Orden de arranque
`restart`	Política de reinicio

YAML: Indentación con **2 espacios**, nunca tabuladores.

Comando	Efecto
`docker compose up -d --build`	Construye y levanta todo en segundo plano
`docker compose ps / logs -f web`	Estado / logs de los servicios
`docker compose exec db bash`	Entra en un servicio
`docker compose down`	Para y elimina contenedores y red
`docker compose down -v`	Además borra los **volúmenes con nombre** (no los bind mounts)